

۱ مقدمه

هدف از این پروژه، طراحی و پیاده‌سازی یک موتور شطرنج با استفاده از یادگیری عمیق و الگوریتم جستجوی درخت مونت کارلو (MCTS) است. این موتور قادر است با استفاده از یک شبکه عصبی کانولوشن، موقعیت‌های صفحه شطرنج را ارزیابی کرده و با ترکیب آن با جستجوی MCTS، سعی کند تا بهترین حرکت را انتخاب کند. داده‌های آموزش از پایگاه داده Lichess دریافت و پس از فیلتر کردن بر اساس ریتینگ بازیکنان، برای آموزش شبکه عصبی استفاده می‌شوند.

۲ معماری شبکه عصبی

شبکه عصبی طراحی شده از نوع کانولوشن با دو خروجی مجزا است: سر سیاست (Policy Head) و سر ارزش (Value Head). این معماری مشابه شبکه‌های استفاده شده در موتورهای معروفی مثل AlphaZero است اما با ابعاد کوچک‌تر برای اجرا روی سخت‌افزار معمولی و برخلاف AlphaZero از Supervised Learning استفاده می‌شود.

۱.۲ ورودی شبکه

ورودی شبکه یک تانسور $8 \times 8 \times 19$ است که هر لایه آن اطلاعات متفاوتی از صفحه شطرنج را نمایش می‌دهد:

- کانال‌های ۰-۵: موقعیت مهره‌های سفید (پیاده، اسب، فیل، رخ، وزیر، شاه)
- کانال‌های ۶-۱۱: موقعیت مهره‌های سیاه
- کانال‌های ۱۲-۱۵: حق قلعه رفتن
- کانال ۱۶: خانه آن پاسان
- کانال ۱۷: نوبت حرکت
- کانال ۱۸: تکرار موقعیت

۲.۲ لایه‌های استخراج ویژگی

شبکه شامل ۴ لایه کانولوشن با مشخصات زیر است:

```
1 self.features = nn.Sequential(  
2     nn.Conv2d(19, 128, 3, padding=1),  
3     nn.BatchNorm2d(128),  
4     nn.ReLU(inplace=True),  
5     nn.Dropout2d(0.3),
```

```

6
7 nn.Conv2d(128, 128, 3, padding=1),
8 nn.BatchNorm2d(128),
9 nn.ReLU(inplace=True),
10 nn.Dropout2d(0.3),
11
12 nn.Conv2d(128, 128, 3, padding=1),
13 nn.BatchNorm2d(128),
14 nn.ReLU(inplace=True),
15 nn.Dropout2d(0.3),
16
17 nn.Conv2d(128, 128, 3, padding=1),
18 nn.BatchNorm2d(128),
19 nn.ReLU(inplace=True),
20 nn.Dropout2d(0.3),
21 )

```

هر لایه کانولوشن از فیلترهای 3×3 با $\text{padding}=1$ استفاده می‌کند که باعث حفظ ابعاد 8×8 صفحه می‌شود. لایه‌های BatchNorm برای نرمال‌سازی و Dropout با نرخ 0.3 برای جلوگیری از بیش‌برازش استفاده شده‌اند.

۳.۲ سر سیاست (Policy Head)

سر سیاست وظیفه پیش‌بینی بهترین حرکت در هر موقعیت را بر عهده دارد:

```

1 self.policy = nn.Sequential(
2 nn.Conv2d(128, 32, 1),
3 nn.BatchNorm2d(32),
4 nn.ReLU(inplace=True),
5 nn.Flatten(),
6 nn.Linear(32 * 64, 4096)
7 )

```

خروجی این سر یک بردار 4096 تایی است که هر عنصر نشان‌دهنده احتمال یک حرکت خاص است. تعداد 4096 از ضرب 64 (خانه مبدأ) در 64 (خانه مقصد) به دست آمده است.

۴.۲ سر ارزش (Value Head)

سر ارزش موقعیت را ارزیابی کرده و یک عدد بین -1 تا 1 برمی‌گرداند که نشان‌دهنده برتری سفید یا سیاه است:

```

1 self.value = nn.Sequential(
2 nn.Conv2d(128, 32, 1),

```

```

3 nn.BatchNorm2d(32),
4 nn.ReLU(inplace=True),
5 nn.Flatten(),
6 nn.Linear(32 * 64, 256),
7 nn.ReLU(inplace=True),
8 nn.Dropout(0.3),
9 nn.Linear(256, 1),
10 nn.Tanh()
11 )

```

۳ ارزیابی هیوریستیک

برای برجسب‌زنی داده‌های آموزش بدون نیاز به موتورهای شطرنج مانند Stockfish، یک تابع ارزیابی هیوریستیک استفاده شده است که عوامل مختلفی را در نظر می‌گیرد:

۱.۳ ارزش مهره‌ها

```

1 piece_values = {
2     chess.PAWN: 100, chess.KNIGHT: 320, chess.BISHOP:
3     330,
4     chess.ROOK: 500, chess.QUEEN: 900, chess.KING: 0
5 }

```

۲.۳ جداول مکان مهره‌ها

برای هر مهره، یک جدول ۸×۸ از امتیازات موقعیتی تعریف شده است. به عنوان مثال، جدول پیاده‌ها:

```

1 pawn_table = [
2 [0, 0, 0, 0, 0, 0, 0, 0],
3 [50, 50, 50, 50, 50, 50, 50, 50],
4 [10, 10, 20, 30, 30, 20, 10, 10],
5 [5, 5, 10, 25, 25, 10, 5, 5],
6 [0, 0, 0, 20, 20, 0, 0, 0],
7 [0, 0, 0, 0, 0, 0, 0, 0],
8 [0, 0, 0, 0, 0, 0, 0, 0],
9 [0, 0, 0, 0, 0, 0, 0, 0]
10 ]

```

۳.۳ سایر عوامل

عوامل دیگری که در ارزیابی در نظر گرفته شده‌اند:

- تحرک مهره‌ها (تعداد حرکات قانونی)
- کنترل مرکز صفحه
- ایمنی شاه و حق قلعه رفتن
- سپر پیاده‌ای جلوی شاه

نهایتاً امتیاز نهایی با تابع $\tanh$ نرمال‌سازی می‌شود:

$$\text{value} = \tanh\left(\frac{\text{score}}{2000}\right)$$

۴ جستجوی درخت مونت کارلو (MCTS)

الگوریتم MCTS با ترکیب شبکه عصبی، جستجوی هوشمندانه‌ای برای یافتن بهترین حرکت انجام می‌دهد. این الگوریتم از ۴ مرحله تشکیل شده است.

۱.۴ مراحل MCTS

۱.۱.۴ انتخاب (Selection)

در این مرحله، با استفاده از فرمول PUTC، بهترین گره برای ادامه جستجو انتخاب می‌شود:

$$\text{score} = Q(s, a) + c \times P(s, a) \times \frac{\sqrt{N(s)}}{1 + N(s, a)}$$

که در آن:

- $Q(s, a)$: ارزش فعلی حرکت (نسبت برد به تعداد بازدیدها)
- $P(s, a)$: احتمال اولیه حرکت از شبکه عصبی
- $N(s)$: تعداد بازدیدهای گره والد
- $N(s, a)$: تعداد دفعاتی که این حرکت انتخاب شده
- c : ثابت اکتشاف (معمولاً ۰.۱)

۲.۱.۴ بسط (Expansion)

اگر گره انتخاب شده قبلاً بسط داده نشده باشد، تمام حرکات قانونی به عنوان فرزندان آن ایجاد می‌شوند. احتمال اولیه هر حرکت از خروجی Head Policy شبکه عصبی گرفته می‌شود.

۳.۱.۴ ارزیابی (Evaluation)

اگر گره به حالت پایان بازی رسیده باشد، نتیجه واقعی بازی برگردانده می‌شود. در غیر این صورت، شبکه عصبی برای ارزیابی موقعیت فراخوانی می‌شود و خروجی Head Value به عنوان ارزش موقعیت در نظر گرفته می‌شود.

۴.۱.۴ بازگشت (Backup)

ارزش به‌دست آمده در تمام گره‌های مسیر جستجو به‌روزرسانی می‌شود:

- تعداد بازدیدها (*visits*) یک واحد افزایش می‌یابد
- مجموع ارزش‌ها (*value_sum*) با ارزش جدید جمع می‌شود
- برای گره‌های مربوط به حریف، علامت ارزش معکوس می‌شود

۲.۴ پیاده‌سازی MCTS

کلاس‌های اصلی MCTS به صورت زیر پیاده‌سازی شده‌اند:

```
1 class MCTSNode:
2     def __init__(self, board, parent=None, move=None,
3         prior=0):
4         self.board = board
5         self.parent = parent
6         self.move = move
7         self.prior = prior
8         self.visits = 0
9         self.value_sum = 0
10        self.children = {}
11        self.is_expanded = False
12
13    def value(self):
14        if self.visits == 0:
15            return 0
16        return self.value_sum / self.visits
17
18    def ucb_score(self, exploration_constant=1.4):
```

```

18         if self.visits == 0:
19             return float('inf')
20         exploration = exploration_constant * self.
21             prior * \
22             math.sqrt(self.parent.visits) / (1 + self.
                visits)
                return self.value() + exploration

```

MCTS: وکلاس اصلی

```

1 class MCTS:
2     def __init__(self, model, num_simulations=800):
3         self.model = model
4         self.num_simulations = num_simulations
5         self.device = torch.device("cuda" if torch.
6             cuda.is_available() else "cpu")
7
8     def search(self, board):
9         root = MCTSNode(board.copy())
10
11         for _ in range(self.num_simulations):
12             node = root
13             path = [node]
14
15             # Selection
16             while node.is_expanded and node.children:
17                 node = self.select_child(node)
18                 path.append(node)
19
20             # Expansion
21             if not node.board.is_game_over() and not
22                 node.is_expanded:
23                 self.expand(node)
24                 node.is_expanded = True
25
26             # Evaluation
27             if node.board.is_game_over():
28                 value = self.get_game_result(node.
29                     board)
30             else:
31                 value = self.evaluate(node.board)
32
33             # Backup
34             for node in reversed(path):

```

```

32         node.visits += 1
33         node.value_sum += value
34         value = -value
35
36     return root

```

۵ آماده‌سازی داده‌ها

داده‌های آموزش از پایگاه داده Lichess دریافت می‌شوند. این پایگاه داده شامل میلیون‌ها بازی شطرنج با فرمت PGN است.

۱.۵ دریافت و فیلتر کردن فایل‌ها

فایل‌های PGN با فرمت فشرده zst از آدرس <https://database.lichess.org/> دریافت می‌شوند. در این پروژه از دو فایل استفاده شده است:

- lichess_db_standard_rated_2015-01.pgn.zst برای آموزش
- lichess_db_standard_rated_2013-01.pgn.zst برای آزمون
- برای فیلتر کردن بر اساس ریتینگ، تابع safe_elo نوشته شده است.

۲.۵ استخراج موقعیت‌ها

از هر بازی، تعدادی موقعیت تصادفی (پیش‌فرض ۱۵ موقعیت) استخراج می‌شود. برای هر موقعیت، موارد زیر ذخیره می‌گردد:

- تنسور صفحه شطرنج (۸×۸×۱۹)
- ارزش موقعیت از تابع ارزیابی هیوریستیک
- حرکتی که در آن موقعیت انجام شده که با عنوان هدف policy از آن استفاده می‌شود.

۳.۵ ذخیره‌سازی دسته‌ای (Batch Saving)

برای جلوگیری از مصرف بیش از حد حافظه، داده‌ها به صورت دسته‌های ۵۰۰۰۰ تایی ذخیره می‌شوند:

```

1 def save_batch(tensors, values, policies, prefix, batch_num
2     ):
3     X = torch.stack(tensors)
4     y_value = torch.tensor(values, dtype=torch.float32)

```

```

4         y_policy = torch.tensor(policies, dtype=torch.long)
5
6         filename = f"{prefix}_batch_{batch_num:04d}.pt"
7         torch.save({
8             'X': X,
9             'y_value': y_value,
10            'y_policy': y_policy,
11            'num_positions': len(tensors)
12        }, filename)

```

۶ آموزش مدل

فرآیند آموزش با استفاده از بهینه‌ساز AdamW و تابع Loss ترکیبی انجام می‌شود.

۱.۶ تابع Loss

تابع Loss از دو بخش تشکیل شده است:

$$\mathcal{L} = \mathcal{L}_{\text{value}} + \mathcal{L}_{\text{policy}}$$

که در آن:

- $\mathcal{L}_{\text{value}} = \text{MSE}(v_{\text{pred}}, v_{\text{target}})$ خطای میانگین مربعات برای سر ارزش
- $\mathcal{L}_{\text{policy}} = \text{CrossEntropy}(p_{\text{pred}}, p_{\text{target}})$ آنترپی متقاطع برای سر سیاست

۲.۶ جلوگیری از بیش‌برازش

برای جلوگیری از بیش‌برازش (Overfitting)، چندین روش استفاده شده است:

۱.۲.۶ Dropout

لایه‌های Dropout با نرخ 0.3 بعد از هر لایه فعال‌سازی قرار داده شده‌اند. Dropout به صورت تصادفی برخی نورون‌ها را در هر مرحله آموزش غیرفعال می‌کند و باعث می‌شود شبکه وابستگی کمتری به نورون‌های خاص پیدا کند.

۲.۲.۶ Decay Weight

در بهینه‌ساز AdamW از پارامتر `weight_decay` با مقدار $1e-4$ استفاده شده است. این کار یک جریمه به وزن‌های بزرگ اضافه می‌کند و باعث ساده‌تر شدن مدل می‌شود.

Stopping Early ۳.۲.۶

در هر epoch بهترین مدل بر اساس کمترین خطای اعتبارسنجی انتخاب می‌گردد و ذخیره می‌شود:

```
1 if avg_val < best_val_loss:
2     best_val_loss = avg_val
3     torch.save(self.model.state_dict(),
4                 f"chess_model_BEST_val_{best_val_loss:.4f}.pth")
```

۷ اجرای برنامه

برنامه دارای یک منوی تعاملی با گزینه‌های زیر است:

Options:

1. Train model
2. Load existing model
3. Play against engine
4. Watch engine vs random
5. Watch engine self-play
6. Exit

۱.۷ آماده‌سازی داده‌ها

برای آماده‌سازی داده‌ها، اسکریپت data.py اجرا می‌شود. این اسکریپت فایل‌های zst را پردازش کرده و فایل‌های دسته‌ای با نام chess_data_batch_*.pt را در ایجاد میکند، که برای یادگیری استفاده می‌شوند.

۲.۷ آموزش مدل

پس از آماده‌سازی داده‌ها، با انتخاب گزینه ۱ از منو و وارد کردن پارامترهای آموزش، فرآیند آموزش آغاز می‌شود. در طول آموزش، مدل‌ها با نام‌های زیر ذخیره می‌شوند:

- chess_model_val_2.3456_epoch_1.pth •
- chess_model_val_2.1234_epoch_2.pth •
- chess_model_BEST_val_1.8765.pth • (بهترین مدل)

۳.۷ بازی با موتور

با انتخاب گزینه‌های ۳ تا ۵ می‌توان با موتور بازی کرد یا آن را در مقابل حرکات تصادفی مشاهده نمود. موتور با انجام ۸۰۰ شبیه‌سازی MCTS برای هر حرکت، بهترین حرکت را انتخاب می‌کند.